

SUPSI

Branch and Bound as PTAS

Studente/i

Tiziano Zamaroni

Sergio Aquilini

Relatore

Monaldo Mastrolilli

Correlatore

-

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

**M-P6050P - Progetto di
Semestre**

Anno

2025

Data

13 dicembre 2025

Indice

1	Introduzione	1
2	Motivazione e Contesto	3
2.1	Motivazione	3
2.2	Contesto	3
3	Algoritmi	5
3.1	Approssimazione soluzione	5
3.2	Branch-and-Bound	5
3.3	Greedy	5
3.4	Profiling	5
4	Requisiti	7
4.1	Requisiti funzionali	7
4.2	Requisiti non funzionali	8
5	Tecnologie utilizzate	9
5.1	Implementazione algoritmi	9
5.2	Analisi dei dati	9
6	Implementation	11
6.1	Architettura e struttura del codice	11
6.2	Branch-and-Bound	11
6.2.1	Implementation Overview	11
6.2.2	Pseudocode	14
6.2.3	Lower-Bound Strategies	15
6.2.4	Rounding Strategies	16
6.2.5	Branching Rules	17
6.3	Greedy	18
6.4	Profiling	18
6.5	Generazione istanze	18

6.6	Analisi dei dati	18
7	Analisi dei risultati	19
8	Metodologia di lavoro	21
9	Conclusione	23
9.1	Risultati ottenuti	23
9.2	Possibili miglioramenti futuri	23

Elenco delle figure

Elenco delle tabelle

1 Introduzione

- Problemi NP-Hard
- Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes
- Unrelated parallel machine scheduling problem
- Uniform/Identical parallel machine scheduling problem (identical caso particolare di uniform)
- Profiling come ulteriore ottimizzazione di identical machine scheduling
- Dimostrazione pratica della validità dell'approccio con profiling

2 Motivazione e Contesto

2.1 Motivazione

2.2 Contesto

3 Algoritmi

3.1 Approssimazione soluzione

Differenze del valore di epsilon tra i vari problemi

3.2 Branch-and-Bound

3.3 Greedy

3.4 Profiling

4 Requisiti

Il lavoro di semestre si inserisce nel contesto del paper *Branch-and-Bound Algorithms as Polynomial-time Approximation Scheme* [1], scritto dal nostro relatore Prof. Monaldo Mastroilli insieme a István Encz Koppányi ed Eleonora Vercesi. Gli autori forniscono un'implementazione Python del Branch-and-Bound per il caso delle *unrelated machines*, comprensiva di euristiche e gestione dei bound. Il nostro compito consiste nel comprendere a fondo tale implementazione, adattarla al caso delle *identical machines* e integrare la tecnica di *profiling*, applicabile poiché le identical machines rappresentano un caso particolare di uniform machines. L'obiettivo finale è verificare sperimentalmente quanto presentato nel paper, confrontando la versione base dell'algoritmo con quella dotata di profiling.

4.1 Requisiti funzionali

Il progetto richiede i seguenti requisiti funzionali:

RF1 – Solver per identical machines

Realizzare una variante dell'algoritmo Branch-and-Bound in grado di risolvere il problema di scheduling su identical machines, adattando la struttura e le funzioni dell'implementazione esistente per le unrelated machines.

RF2 – Integrazione del profiling

Implementare il meccanismo di profiling descritto nel paper, così da ottenere una seconda variante dell'algoritmo direttamente confrontabile con la versione base.

RF3 – Generazione ed esecuzione degli esperimenti

Eseguire entrambe le varianti dell'algoritmo e registrare i risultati, raccogliendo una quantità consona di dati.

RF4 – Raccolta dei dati

Memorizzare in formato strutturato informazioni quali nodi esplorati, tempo di esecuzione, delta rispetto alla soluzione ottima dell'istanza considerata e altre metriche rilevanti.

RF5 – Validazione sperimentale

Analizzare i dati raccolti, con l'aiuto di strumenti statistici e di visualizzazione, per verificare l'efficacia del profiling rispetto alla versione base dell'algoritmo.

4.2 Requisiti non funzionali

Non erano previsti requisiti non funzionali formali, ma il progetto implicava alcuni vincoli pratici:

Efficienza computazionale

L'implementazione e la scelta delle istanze dovevano permettere di generare e analizzare un numero significativo di dati entro i limiti temporali del lavoro di semestre.

Ripetibilità

La generazione delle istanze e l'esecuzione degli algoritmi dovevano essere riproducibili tramite seed controllati.

Mantenibilità e pulizia del codice

Anche trattandosi di un esperimento non destinato a evoluzione futura, il codice doveva essere scritto in modo chiaro e integrarsi coerentemente con la struttura esistente.

5 Tecnologie utilizzate

5.1 Implementazione algoritmi

L'implementazione degli algoritmi è stata sviluppata in *Python* a partire dal codice fornito dagli autori del paper [1], originariamente progettato per il caso delle *unrelated machines*. In tale versione, il lower bound dei nodi del Branch-and-Bound veniva calcolato tramite il solver *SCIP* [2], utilizzato attraverso la libreria *PySCIPOpt*. Questa scelta, appropriata per il problema generale, consente agli autori di ottenere bound frazionari mediante *binary search* e *linear relaxation*.

Nel nostro progetto, tuttavia, tale approccio si è rivelato eccessivamente oneroso e non strettamente necessario. Nel caso delle *identical machines*, infatti, la struttura del problema è significativamente più semplice e permette l'impiego di metodi più leggeri per il calcolo del lower bound. Inoltre, l'utilizzo di *SCIP* [2] presentava un ulteriore limite pratico: non è stato possibile forzare il solver a utilizzare sistematicamente il metodo *primal simplex*, condizione necessaria per garantire che il numero di variabili frazionarie nella soluzione LP rimanesse inferiore o uguale al numero di macchine. L'assenza di tale garanzia comprometteva l'affidabilità della fase di arrotondamento e introduceva un overhead computazionale non trascurabile.

Per queste ragioni, abbiamo sostituito integralmente l'utilizzo di *SCIP* [2] con un metodo di calcolo del lower bound basato su un criterio *greedy*, pienamente sufficiente per il caso delle *identical machines* e decisamente più efficiente dal punto di vista computazionale. Questa scelta ha permesso di ridurre drasticamente il tempo di elaborazione dei nodi del Branch-and-Bound, mantenendo comunque una qualità del bound adeguata agli scopi del progetto.

5.2 Analisi dei dati

L'analisi dei dati sperimentali è stata effettuata in *Python* utilizzando le principali librerie per la data science. *numpy* è stato impiegato per l'elaborazione numerica di base, *pandas* per l'organizzazione e l'aggregazione dei risultati raccolti durante gli esperimenti, e *matplotlib* per la generazione delle visualizzazioni utilizzate nel confronto tra la versione base

dell'algoritmo e quella dotata di profiling. I dati sono stati esportati in formato CSV e analizzati tramite notebook dedicati, permettendo di sintetizzare l'andamento delle performance e supportare quantitativamente la discussione dei risultati.

6 Implementation

6.1 Architettura e struttura del codice

Still needed??

6.2 Branch-and-Bound

The implementation employed in this project is based on a branch-and-bound (B&B) framework originally developed for the *unrelated machines scheduling* problem. The same structure is later reused for the identical machines case, where only the lower-bound routines and selected optimisation components (such as profiling) are adapted, while the core branching, bounding, rounding, and node-selection logic remains unchanged.

6.2.1 Implementation Overview

Each node of the branch-and-bound tree represents a subproblem corresponding to a partial assignment of jobs to machines and is characterised by:

- a set of fixed assignments $\text{fixed} \subseteq \{0, \dots, n-1\} \times \{0, \dots, m-1\}$,
- an overhead vector $h \in \mathbb{R}^m$, where h_i denotes the load already assigned to machine i ,
- a fractional solution X^{frac} obtained from a relaxation of the problem,
- a node lower bound LB and a node upper bound UB,
- the depth of the node in the search tree.

The global state maintains a best-known feasible solution, represented by a global upper bound LUB, and a global lower bound LLB derived from the set of active nodes in the search tree.

The behaviour of the algorithm is governed by four main components.

Bounding. For each node, a lower bound on the optimal makespan of the associated subproblem is computed by solving either a linear relaxation or a feasibility-based binary search (see Section 6.2.3). Nodes whose lower bound is not strictly smaller than the current global upper bound LUB are pruned.

Branching. When the relaxation at a node yields a non-integer solution, one fractional job is selected according to a branching rule. This job is fixed on each possible machine, generating up to m child nodes and partitioning the remaining search space.

Rounding. A feasible integer solution is obtained from the fractional relaxation through a deterministic rounding strategy. The resulting makespan defines a node upper bound UB, which may improve the current global best solution.

Node selection. Active nodes are stored in a priority queue and extracted according to a configurable selection strategy (e.g., smallest lower bound first, depth-first, or breadth-first). The choice of strategy affects the traversal order of the search tree but not the correctness of the algorithm.

The algorithm terminates when the relative optimality gap satisfies

$$\frac{\text{LUB}}{\text{LLB}} < 1 + \varepsilon,$$

indicating that the incumbent solution is within a factor $(1 + \varepsilon)$ of optimality, or when the search space has been fully explored. In practice, an additional limit on the maximum number of explored nodes may be enforced to prevent excessive runtimes.

6.2.2 Pseudocode

Algorithm 1 Branch-and-bound framework (unrelated machines scheduling)

Require: instance $p_{j,i}$, tolerance ε ; strategies for node selection, bounding, branching, rounding

```

1:  $(X^{\text{frac}}, \text{LB}) \leftarrow \text{LowerBound}(p, h=0, \text{fixed}=\emptyset)$ 
2: if  $X^{\text{frac}}$  is integral then return  $(X^{\text{frac}}, \text{LB})$ 
3: end if
4:  $(X^*, \text{LUB}) \leftarrow \text{Rounding}(X^{\text{frac}}, \emptyset)$ 
5: Initialize queue  $Q$  with root node ( $\text{fixed}=\emptyset, h=0, X^{\text{frac}}, \text{LB}$ )
6:  $\text{LLB} \leftarrow \text{LB}$ 
7: while  $Q \neq \emptyset$  do
8:    $\text{LLB} \leftarrow \min(\text{LUB}, \min_{N \in Q} N.\text{LB})$ 
9:   if  $\text{LUB}/\text{LLB} < 1 + \varepsilon$  then break
10:  end if
11:  Extract node  $N$  from  $Q$  according to the node selection strategy
12:  Select a fractional job  $j$  from  $N.X^{\text{frac}}$  (branching rule)
13:  for  $q \in \{0, \dots, m-1\}$  do ▷ create child by fixing  $(j, q)$ 
14:     $\text{fixed}' \leftarrow N.\text{fixed} \cup \{(j, q)\}, \quad h' \leftarrow N.h \text{ with } h'_q \leftarrow h_q + p_{j,q}$ 
15:    if  $\text{fixed}'$  assigns all jobs then
16:       $\text{UB}' \leftarrow \max_i h'_i$ 
17:      if  $\text{UB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (\text{fixed}', \text{UB}')$ 
18:      end if
19:      continue
20:    end if
21:     $(X^{\text{frac}'}, \text{LB}') \leftarrow \text{LowerBound}(p, h', \text{fixed}')$ 
22:    if  $\text{LB}' \geq \text{LUB}$  then continue ▷ prune by bound
23:    end if
24:    if  $X^{\text{frac}'}$  is integral then
25:      if  $\text{LB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (X^{\text{frac}'}, \text{LB}')$ 
26:      end if
27:      continue
28:    end if
29:     $(X^{\text{int}'}, \text{UB}') \leftarrow \text{Rounding}(X^{\text{frac}'}, \text{fixed}')$ 
30:    if  $\text{UB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (X^{\text{int}'}, \text{UB}')$ 
31:    end if
32:    Insert child node  $(\text{fixed}', h', X^{\text{frac}'}, \text{LB}')$  into  $Q$ 
33:  end for
34: end while
35: return  $(X^*, \text{LUB})$ 

```

Algorithm 1 summarises the inherited branch-and-bound framework used for the unrelated machines scheduling problem. For each node, a relaxation is solved to obtain a fractional solution X^{frac} and a lower bound LB on the optimal makespan of the corresponding subproblem.

A global upper bound LUB is maintained through feasible integer solutions produced either by rounding fractional solutions (Section 6.2.4) or by integrality of the relaxation. Whenever a solution with smaller makespan is found, the incumbent solution X^* and the global upper bound are updated.

The search space is explored by repeatedly extracting nodes from a priority queue according to the selected node selection strategy. If the extracted node is fractional, a branching rule selects one fractional job j ; branching fixes j on each machine, generating up to m child nodes. For each child node, a new lower bound LB' is computed and the node is immediately discarded if $LB' \geq LUB$, since it cannot lead to an improvement over the incumbent solution. The global lower bound LLB is derived from the minimum lower bound among the nodes currently stored in the queue, clipped by LUB.

6.2.3 Lower-Bound Strategies

The inherited implementation provides two alternative strategies to compute node-level lower bounds within the branch-and-bound framework. Both operate on the subproblem defined by:

- the set of fixed assignments *fixed*,
- the overhead vector $h \in \mathbb{R}^m$,
- the remaining unfixed jobs.

In both cases, the output consists of a fractional assignment X^{frac} and a scalar lower bound LB.

Linear relaxation (lin_relax). In the linear-relaxation strategy, a linear program is solved in which each unfixed job may be fractionally assigned to machines. Let $x_{j,i} \geq 0$ denote the fraction of job j assigned to machine i , and let $C_{\max}$ denote the makespan. The model can be expressed as:

$$\begin{aligned}
 & \min C_{\max} \\
 & \text{s.t. } \sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed,} \\
 & \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \leq C_{\max} - h_i && \forall i = 0, \dots, m-1, \\
 & x_{j,i} \geq 0 && \forall j, i.
 \end{aligned}$$

The fixed jobs do not appear as decision variables; their contribution is encoded in the overhead h . The optimal value $C_{\max}^*$ is used as the node lower bound LB, and the corresponding solution defines X^{frac} . Due to the structure of the formulation, at most m jobs are fractional in the optimal solution.

Binary search on the makespan (bin_search). The second strategy performs a binary search on the makespan value. An interval $(\ell, r]$ is maintained such that ℓ is infeasible and r is feasible. For a candidate makespan C , a feasibility LP is solved with variables $x_{j,i}$ for unfixed jobs, subject to:

$$\begin{aligned}
 \sum_{i=0}^{m-1} x_{j,i} &= 1 && \forall j \text{ unfixed,} \\
 \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} &\leq C - h_i && \forall i = 0, \dots, m-1, \\
 x_{j,i} &= 0 && \text{if } p_{j,i} > C - h_i, \\
 x_{j,i} &\geq 0 && \text{otherwise.}
 \end{aligned}$$

If the LP is feasible, the value C becomes the new right endpoint; otherwise it becomes the new left endpoint. The process continues until $(\ell, r]$ cannot be further refined, and the final feasible value r is taken as the node lower bound. The corresponding LP solution defines the fractional assignment X^{frac} .

6.2.4 Rounding Strategies

When the relaxation at a node yields a fractional solution X^{frac} , a deterministic rounding procedure is applied to construct a feasible integer assignment. The resulting schedule defines a node upper bound UB, which may improve the current global upper bound LUB.

The rounding implementation operates on the subproblem defined by the fixed assignments fixed and the overhead vector $h \in \mathbb{R}^m$. Jobs already fixed do not appear as decision variables; their contribution is accounted for by the overhead values h_i . Moreover, jobs that are already integrally assigned in X^{frac} are preserved.

Let F denote the set of *fractional jobs*, i.e., those jobs j such that $x_{j,i}$ is non-integer for at least one machine i . The implementation assumes $|F| \leq m$, which is consistent with the structure of the relaxations used in Section 6.2.3. Rounding proceeds by first computing the machine completion times induced by (i) integrally assigned jobs in X^{frac} and (ii) fixed jobs, and then assigning the jobs in F according to one of the following strategies.

Assign-to-shortest machine (`all_to_shortest`). Each fractional job $j \in F$ is assigned to the machine that minimises its processing time:

$$i^*(j) \in \arg \min_{i=0, \dots, m-1} p_{j,i}.$$

The completion times are updated accordingly, and the upper bound is obtained as the makespan of the resulting schedule:

$$\text{UB} = \max_{i=0, \dots, m-1} C_i.$$

This strategy is computationally inexpensive and purely greedy with respect to individual job processing times.

Best matching of fractional jobs (`best_matching`). Since $|F| \leq m$, the fractional jobs can be assigned by enumerating all injective mappings of F to machines. Equivalently, all permutations of machines of length $|F|$ are considered. For each candidate placement, the completion times are updated and the makespan is evaluated; the assignment yielding the minimum makespan among the enumerated placements is selected, and the corresponding makespan is returned as UB.

6.2.5 Branching Rules

When the relaxation at a node produces a non-integer solution, a set of *fractional jobs* is obtained, namely those jobs j for which at least one variable $x_{j,i}$ is non-integer. The implementation assumes that the number of such jobs is at most the number of machines m , which is guaranteed by the structure of the relaxations. A branching rule selects one job j from this set, and the node is branched by fixing job j on each machine $q = 0, \dots, m-1$, generating up to m children.

Two selection criteria are provided.

Maximum minimum processing time (`max_min_proc`). For each fractional job j , the quantity

$$\min_{i=0, \dots, m-1} p_{j,i}$$

is considered. The branching job is the one that maximises this minimum processing time. This rule gives priority to jobs that are relatively expensive even on their best machine, so that branching decisions focus on jobs with high potential impact on the makespan.

Maximum average processing time (`max_avg_proc`). For each fractional job j , the average processing time

$$\frac{1}{m} \sum_{i=0}^{m-1} p_{j,i}$$

is computed. The branching job is the one with maximum average processing time, i.e., the job that is globally heaviest across all machines. This tends to branch first on jobs that dominate the total processing workload.

6.3 Greedy

6.4 Profiling

6.5 Generazione istanze

6.6 Analisi dei dati

7 Analisi dei risultati

- Overview istanze e dati raccolti
- Istanze relativamente piccole ma con diversi seed
- Difficoltà variabile a dipendenza del seed, a parità di dimensione
- Statistiche no-profiling vs. prune per tipo di istanza
- Evidenziare che profiling rimane percorribile quando la versione base non arriva a soluzione

8 Metodologia di lavoro

- Suddivisione dei compiti
- Pianificazione e Task
- Pair programming / brainstorming
- Weekly
- Raccolta dei dati

9 Conclusione

9.1 Risultati ottenuti

9.2 Possibili miglioramenti futuri

Elenco di possibili sviluppi futuri:

- abc
- def

Bibliografia

- [1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.
- [2] Scip optimization suite. <https://scipopt.org>.